

Updating with Relationship

Contents

- More relationship querying
 - `whereHas()` / `orWhereHas()`
 - `whereDoesntHave()` / `orWhereDoesntHave()`
- Updating with relationship
- Directive `@selected` and `@checked`
- Example
- References

More relationship querying

Querying is based on other model value

- In some cases, we want to query the models that are based on other related models value.
 - For example:
 - We want to list all `Shop` models that contain `Product` models have price > 100.
 - Or we want to list all `Product` models that have category code start with CT1.
- Eloquent provides the following methods to do that:
 - `whereHas()` / `orWhereHas()`
 - `whereDoesntHave()` / `orWhereDoesntHave()`

Example — Product model

```
// Product  
  
function category(): BelongsTo {  
    return $this->belongsTo(Category::class);  
}  
  
function shops(): BelongsToMany {  
    return $this->belongsToMany(Shop::class);  
}
```

Example — Category and Shop models

```
// Category
```

```
function products(): HasMany {  
    return $this->hasMany(Product::class);  
}
```

```
// Shop
```

```
function products(): BelongsToMany {  
    return $this->belongsToMany(Product::class);  
}
```

whereHas () / orWhereHas ()

- Using

- `whereHas('relation_name', $closure_to_query_related_model)`
- `$closure_to_query_related_model` provides query object of related model as function parameter.
- return query object.

- Add `where exists` with the given query in

`$closure_to_query_related_model`.

- `orWhereHas ()` is same as `whereHas ()` but join with OR operator.

whereHas () / orWhereHas () — Example

- List all products that have category code start with CT1.

```
// import Illuminate\Database\Eloquent\Builder
$products = Product::whereHas('category', function(Builder $query) {
    // $query is for Category model
    $query->where('code', 'LIKE', 'CT1%');
})->get();
```

- The previous code will generate SQL is like:

```
select * from `products` where exists
(select * from `categories` where
    `products`.`category_id` = `categories`.`id` and `code` LIKE 'CT1%')
```

whereDoesntHave () / orWhereDoesntHave ()

- Using

- `whereDoesntHave('relation_name', $closure_to_query_related_model)`
- `$closure_to_query_related_model` provides query object of related model as function parameter.
- return query object.

- Add where **not** exists with the given query in
`$closure_to_query_related_model`.

- `orWhereDoesntHave ()` is same as `whereDoesntHave ()` but join with OR operator.

whereDoesntHave() / orWhereDoesntHave() — Example

- List all shops that **do not** contain product with code PD001.

```
// import Illuminate\Database\Eloquent\Builder
$shops = Shop::whereDoesntHave('products', function(Builder $query) {
    // $query is for Product model
    $query->where('code', 'PD001');
})->get();
```

- The previous code will generate SQL is like:

```
select * from `shops` where not exists
(select * from `products` inner join `product_shop`
on `products`.`id` = `product_shop`.`product_id` where
`shops`.`id` = `product_shop`.`shop_id` and `code` = 'PD001')
```

Updating with relationship

Create related model — `save()`

- We can use method `save()` from relationship query to create the related models.

```
$product = new Product();  
$product->code = 'PD001';  
$product->name = 'Book';
```

```
$category = Category::where('code', 'CT001')->firstOrFail();
```

```
$category->products()->save($product);
```

- Eloquent automatically assign `category_id` to Product model.

Create related model — `saveMany()`

- We can create multiple related models by using method `saveMany()`.

```
$product1 = new Product();  
$product1->code = 'PD001';  
$product1->name = 'Book';
```

```
$product2 = Product::where('code', 'PD002')->firstOrFail();
```

```
$category = Category::where('code', 'CT001')->firstOrFail();
```

```
$category->products()->saveMany([$product1, $product2]);
```

Create related model — `create()` and `createMany()`

- For Mass Assignment, query relationship also support `create()` and `createMany()`.

```
$category = Category::where('code', 'CT001')->firstOrFail();  
$category->products()->createMany([  
    ['code' => 'PD001', 'name' => 'Book'],  
    ['code' => 'PD002', 'name' => 'Pencil'],  
]);
```

- But the related model have to be set up with Mass Assignment, `# [Fillable(...)]`.

Recursively Saving Models — `push()`

- If we would like to save our model and all its associated relationships, we may use the method `push()`.

```
$category = Category::where('code', 'CT001')->firstOrFail();  
$category->products[0]->name = 'The new Book';  
$category->products[1]->name = 'The new pencil';  
  
/* The following statement not only saves category data  
   but also saves the first and the second product. */  
$category->push();
```

BelongsTo Relationship — `associate()`

- If we would like to assign a child model, has relationship `BelongsTo`, to a new parent model, we may use the method `associate()`.
 - **Don't set** to field `table_name_id` directly, it **breaks the internal process** of Eloquent.

```
$product = Product::where('code', 'PD001')->firstOrFail();  
$category = Category::where('code', 'CT003')->firstOrFail();
```

```
// The following code will change $product to category CT003.  
$product->category()->associate($category);  
$product->save(); // save() is required
```

- And we can **just use an id** of category, assume category CT003 has `id = 3`.

```
$product->category()->associate(3);  
$product->save(); // save() is required
```

BelongsTo Relationship — `dissociate()`

- To remove a parent model from a child model, we may use the method `dissociate()`.
 - **The foreign key** has to be set to **nullable**.

```
$product = Product::where('code', 'PD001')->firstOrFail();
```

```
// The following code will remove category from $product.
```

```
$product->category()->dissociate();
```

```
$product->save(); // save() is required
```

BelongsToMany — attach()

- Eloquent also provides methods to make working with *many-to-many* relationships more convenient.
 - For example, we want to add Product model to Shop model then we use method `attach()`.

```
$shop = Shop::where('code', 'SH001')->firstOrFail();  
$product1 = Product::where('code', 'PD001')->firstOrFail();  
$product2 = Product::where('code', 'PD002')->firstOrFail();  
$product3 = Product::where('code', 'PD003')->firstOrFail();
```

```
$shop->products()->attach($product1); // attach single model  
$shop->products()->attach([$product2, $product3]); // attach multiple models
```

BelongsToMany — detach()

- And we can remove Product model from Shop model by using method detach().

```
$shop = Shop::where('code', 'SH001')->firstOrFail();  
$product1 = $shop->products[0];  
$product2 = $shop->products[1];  
$product3 = $shop->products[2];
```

```
$shop->products()->detach($product1); // detach single model  
$shop->products()->detach([$product2, $product3]); // detach multiple models
```

BelongsToMany — sync()

- When we want `attach()` and `detach()` on the same time we use method `sync()`.
 - Any models that are not in the given array will be removed from the intermediate table.

```
$shop = Shop::where('code', 'SH001')->firstOrFail();  
$product1 = Product::where('code', 'PD001')->firstOrFail();  
$product2 = Product::where('code', 'PD002')->firstOrFail();  
  
$shop->products()->sync([$product1, $product2]);  
// Now $shop have only 2 product PD001 and PD002.
```

Refresh model with data from database.

- When we call method `attach()`, `detach()` or `sync()`, Eloquent just updates the data in database.
 - That means our **in-memory models do not get updating**.
- If we want to use the related models after those methods were called, we have to refresh our models with the data from database by using method **`refresh()`**.

```
$product = Product::where('code', 'PD001')->firstOrFail();  
$shop = Shop::where('code', 'SH001')->firstOrFail();  
$product->shops()->attach($shop);  
// Only calling  
// $product->shops[count($product->shops) - 1];  
// would not get the attached shop,  
// we have to call $product->refresh() first.  
$product->refresh();  
$attached_shop = $product->shops[count($product->shops) - 1];
```

Using `id` instead of entire model

- Same as `associate()`, many of Eloquent updating relationship methods allow us to **use `id` instead of entire model**.
- So, `attach()`, `detach()` and `sync()` can work with `id`.

```
$shop->products()->attach(1);           // add product id = 1 to $shop  
$shop->products()->attach([2, 3]); // add product id = 2 and 3 to $shop
```

Directive

@selected and @checked

Assign old value to controls

- Consider the **updating page**, we assign the old value to controls, e.g.

```
<input type="text" name="code" value="{{ $product->code }}" />
```

- But for controls like drop-down list, checkbox or radio button, we have to assign `selected` or `checked` attribute to the old option, e.g.

- The generated HTML would be like:

```
<!-- Assume the old value of category is 'CT002' -->
<select name="category">
    <option value="CT001">[CT001] PHP</option>
    <option value="CT002" selected>[CT002] JavaScript</option>
    <option value="CT003">[CT003] Python</option>
</select>
```

Assign selected or checked in Blade templates

- For Blade template, we have to compare drop-down value with old value to **assign selected attribute** to the related `<option>`, e.g.

- With `@if` directive:

```
<select name="category">
@foreach($categories as $category)
    <option value="{{ $category->code }}"
        @if($product->category->code === $category->code) selected @endif
    >[{{ $category->code }}] {{ $category->name }}</option>
@endforeach
</select>
```

- or by using ternary operator

```
<select name="category">
@foreach($categories as $category)
    <option value="{{ $category->code }}"
        {{ ($product->category->code === $category->code)? 'selected' : '' }}
    >[{{ $category->code }}] {{ $category->name }}</option>
@endforeach
</select>
```

Laravel 9+, @selected and @checked

- From Laravel 9, it provides the shorter syntax @selected and @checked.
- Using
 - @selected(*expression*)
 - @checked(*expression*)
- if *expression* is true, it will generate selected or checked word respectively.

@selected Example

- From the previous example, we use @selected instead, e.g.

```
<select name="category">
  @foreach($categories as $category)
    <option value="{{ $category->code }}"
      @selected($product->category->code === $category->code)
      >[{{ $category->code }}] {{ $category->name }}</option>
  @endforeach
</select>
```

Example

- Reuse project from week-09.
- Add shop to product
- Remove shop from product

Example — Add/Remove Shop in Product

- For **adding Shop** to Product:
 - Method **get** on `/products/{product}/shops/add` — **show adding form**.
 - Method **post** on `/products/{product}/shops` — **add selected shop to product**.
- For **removing Shop** from Product:
 - Method **post** on `/products/{product}/shops/remove` — **remove selected shop from product**.

Example — Add Shop to Product

DB - Product: PD001: Shops

localhost:8000/products/PD001/shops

Product Shop Category

Product: PD001: Shops

Search
Term

Filter Clear

< Back Add Shops

List of Shops for Programming PHP

Code	Name	Owner	No. of Products	
SH001	Pachara Shop	Pachara	2	Remove
SH002	Phasit Shop	Phasit	3	Remove

© Copyright Pachara's Database.

DB - Product: PD001: Add Shop

localhost:8000/products/PD001/shops/add

Product Shop Category

Product: PD001: Add Shops

Search
Term

Filter Clear

< Back < 1 2 3 >

List of Shops for Adding to Programming PHP

Code	Name	Owner	No. of Products	
SH003	Other Shop	Pachara	3	Add
SH101	Adam Shop	Adam	3	Add
SH102	MadA Shop	Adam	1	Add
SH103	Page Turners	Bob	0	Add
SH104	Literary Lounge	Cindy	0	Add

© Copyright Pachara's Database.

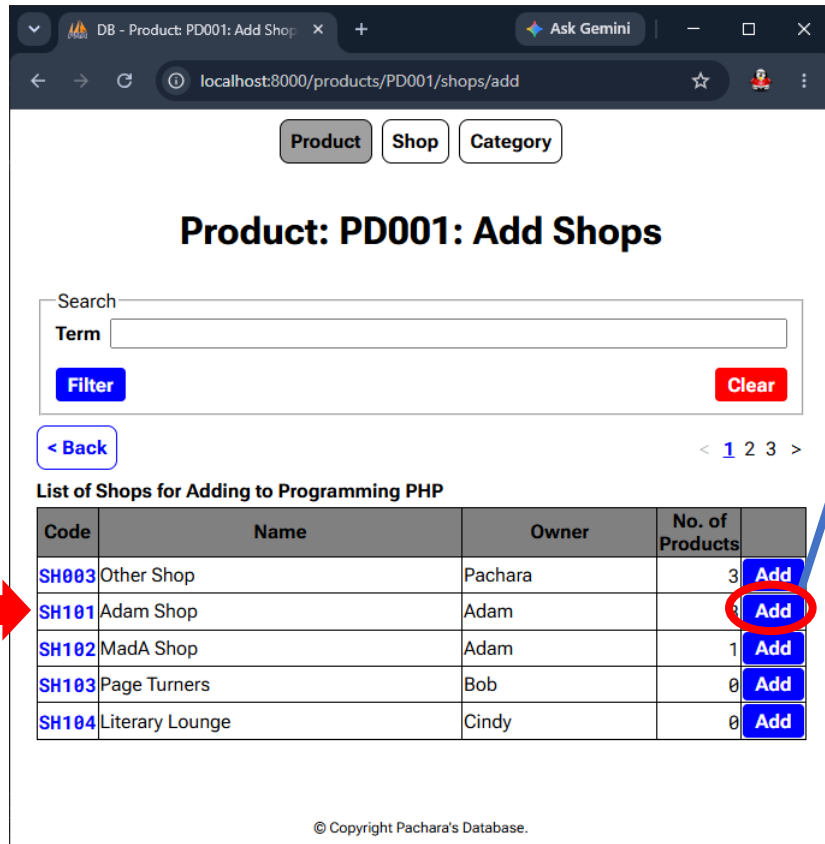
Update route `products.view-shops`

- Add link to route `products.add-shops-form`.

In route `products.add-shops-form`

- List all shops that **do not contain** the given product.
 - E.g., SH001 and SH002 **do not show** in this list.
- Add back link to `products.view-shops`.

Example — Add Shop to Product (cont.)



Product Shop Category

Product: PD001: Add Shops

Search
Term
Filter Clear

< Back < 1 2 3 >

List of Shops for Adding to Programming PHP

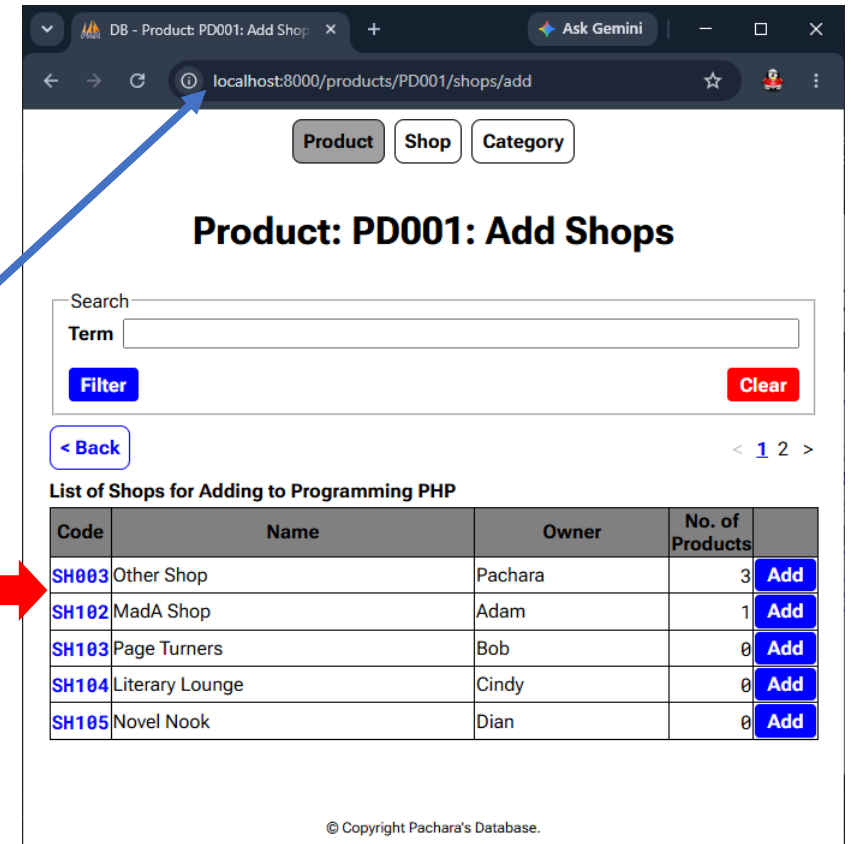
Code	Name	Owner	No. of Products	
SH003	Other Shop	Pachara	3	Add
SH101	Adam Shop	Adam	1	Add
SH102	MadA Shop	Adam	1	Add
SH103	Page Turners	Bob	0	Add
SH104	Literary Lounge	Cindy	0	Add

© Copyright Pachara's Database.

User click add to submit form-data to route `products.add-shop`.

`products.add-shop`

Route `products.add-shop` **add submitted shop** to product
- Then **redirect back** to route `products.add-shops-form`.



Product Shop Category

Product: PD001: Add Shops

Search
Term
Filter Clear

< Back < 1 2 >

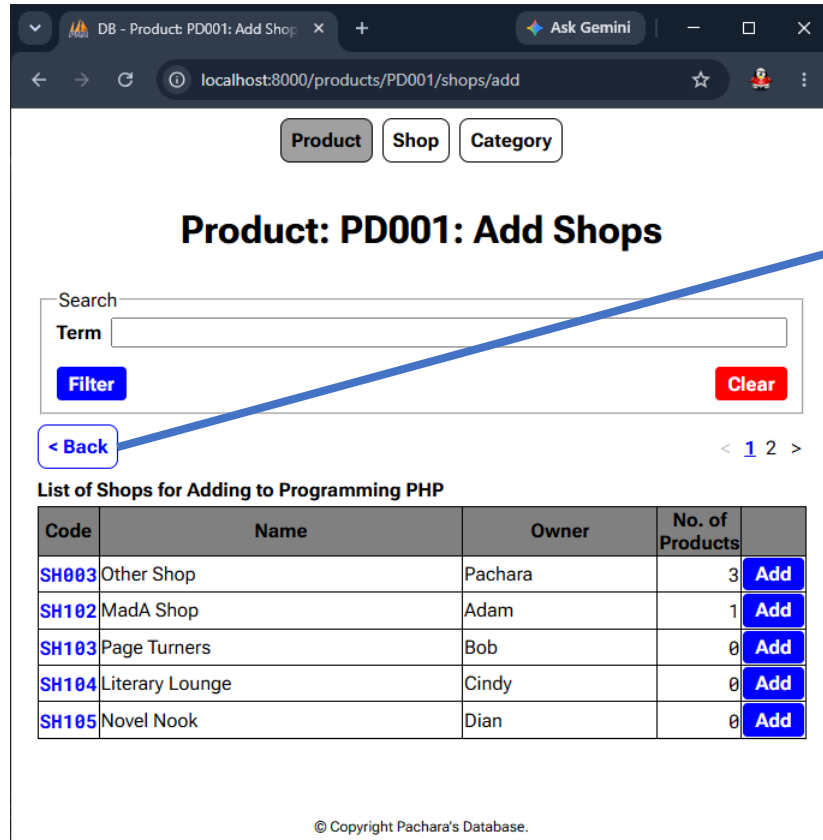
List of Shops for Adding to Programming PHP

Code	Name	Owner	No. of Products	
SH003	Other Shop	Pachara	3	Add
SH102	MadA Shop	Adam	1	Add
SH103	Page Turners	Bob	0	Add
SH104	Literary Lounge	Cindy	0	Add
SH105	Novel Nook	Dian	0	Add

© Copyright Pachara's Database.

In this case shop SH101 **does not appear** in list because we had added it to product.

Example — Add Shop to Product (cont.)



DB - Product: PD001: Add Shop

localhost:8000/products/PD001/shops/add

Product Shop Category

Product: PD001: Add Shops

Search
Term

Filter Clear

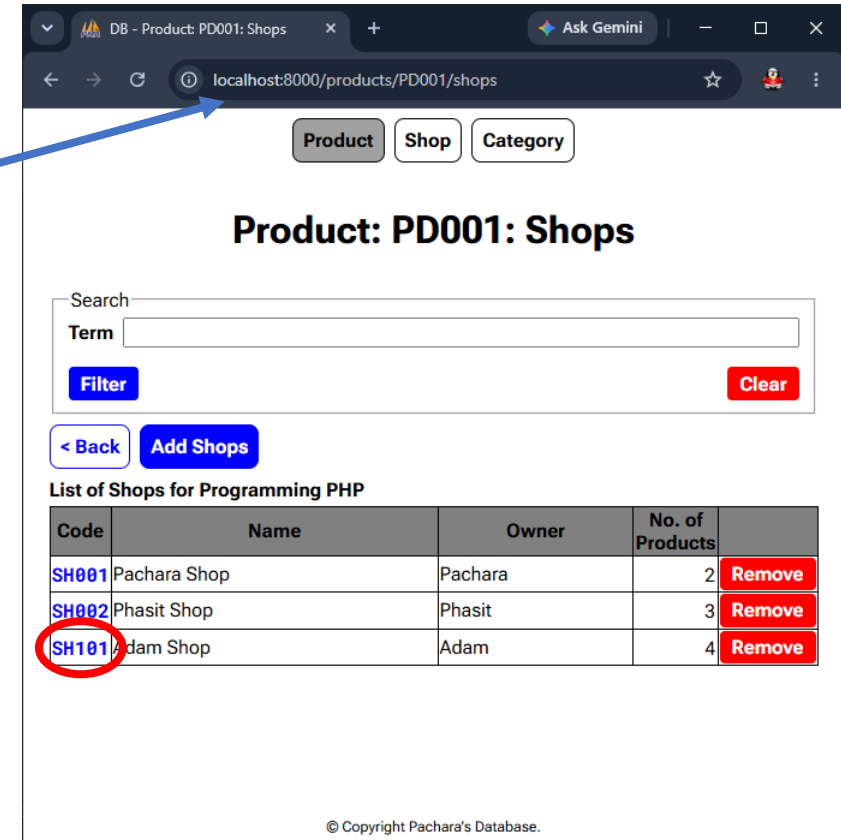
< Back < 1 2 >

List of Shops for Adding to Programming PHP

Code	Name	Owner	No. of Products	
SH003	Other Shop	Pachara	3	Add
SH102	MadA Shop	Adam	1	Add
SH103	Page Turners	Bob	0	Add
SH104	Literary Lounge	Cindy	0	Add
SH105	Novel Nook	Dian	0	Add

© Copyright Pachara's Database.

Then link back to route `products.view-shops`.



DB - Product: PD001: Shops

localhost:8000/products/PD001/shops

Product Shop Category

Product: PD001: Shops

Search
Term

Filter Clear

< Back Add Shops

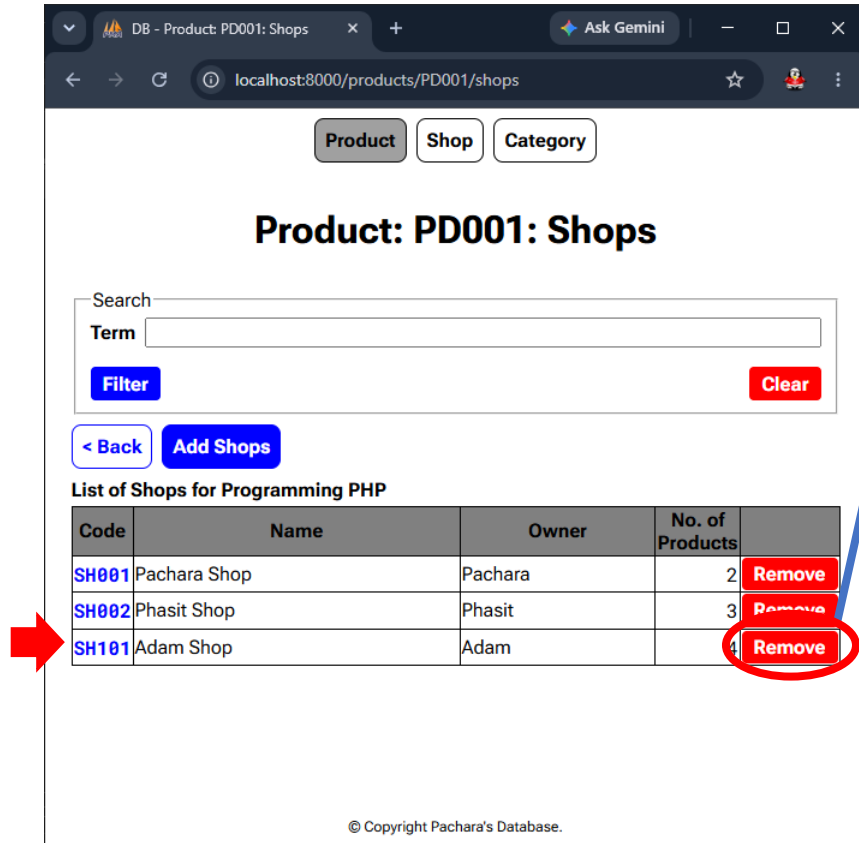
List of Shops for Programming PHP

Code	Name	Owner	No. of Products	
SH001	Pachara Shop	Pachara	2	Remove
SH002	Phasit Shop	Phasit	3	Remove
SH101	Adam Shop	Adam	4	Remove

© Copyright Pachara's Database.

Now shop SH101 is **already in shop list** of product PD001.

Example — Remove Shop from Product



Product Shop Category

Product: PD001: Shops

Search
Term
Filter Clear

< Back Add Shops

List of Shops for Programming PHP

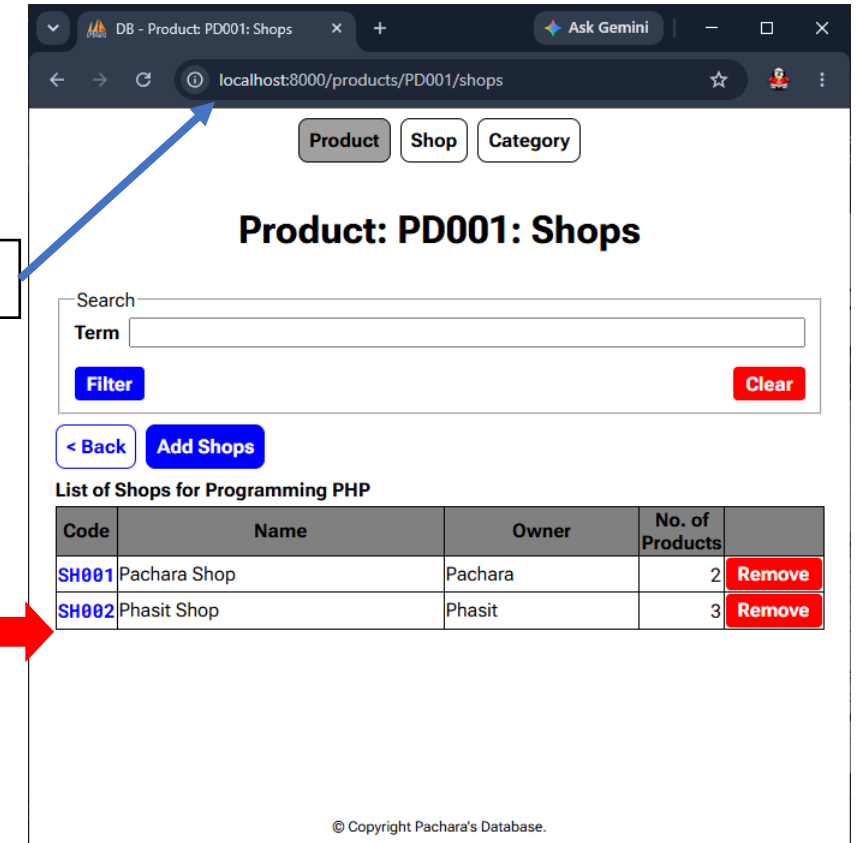
Code	Name	Owner	No. of Products	
SH001	Pachara Shop	Pachara	2	Remove
SH002	Phasit Shop	Phasit	3	Remove
SH101	Adam Shop	Adam	1	Remove

© Copyright Pachara's Database.

User click add to submit form-data
to route `products.remove-shop`.

`products.remove-shop`

Route `products.remove-shop`
remove the submitted shop form
product
- Then **redirect back** to route
`products.view-shops`.



Product Shop Category

Product: PD001: Shops

Search
Term
Filter Clear

< Back Add Shops

List of Shops for Programming PHP

Code	Name	Owner	No. of Products	
SH001	Pachara Shop	Pachara	2	Remove
SH002	Phasit Shop	Phasit	3	Remove

© Copyright Pachara's Database.

Now shop SH101 is **already
removed from list**

Example — Add Filter out Shop by Product

1. In ProductController

- When we want to add shops for product, we need to make sure that **shops don't have this product**.
- So, we add method for **filtering them out**.

```
function filterOutShopByProduct(  
    Builder|Relation $shopQuery,  
    Product $product,  
): void {  
    /* We use whereDoesntHave() for getting shops that do not  
     * contain this product, in this case, we use code for checking.  
     */  
    $shopQuery->whereDoesntHave(  
        'products',  
        static function (Builder $productQuery) use ($product): void {  
            $productQuery->where('code', $product->code);  
        },  
    );  
}
```

Example — products.add-shops-form

2. In ProductController

- We add `showAddShopsForm()` for showing adding shops form.

```
class ProductController extends SearchableController
{
    // Now we add showAddShopsForm() method
    function showAddShopsForm(
        string $productCode,
        ServerRequestInterface $request,
    ): View {
        // method body is on the next pages
    }
}
```

Example — products.add-shops-form (cont.)

2. In ProductController (cont.)

- In method `showAddShopsForm()` body.

```
$product = $this->find($productCode);
$shopController = \resolve(ShopController::class);
$criteria = $shopController->prepareCriteria($request->getQueryParams());

$shopQuery = $shopController->getQuery()->withCount('products');
$this->filterOutShopByProduct($shopQuery, $product);

return view('products.add-shops-form', [
    'criteria' => $criteria,
    'product' => $product,
    'shops' => $shopQuery->paginate($shopController::MAX_ITEMS),
]);
```

Example — products.add-shop

3. In ProductController

- We use `attach()` method to add shop to product.
- We use `redirect() -> back()` **instead of** `redirect() -> route()` for keeping out query string for searching.
 - `redirect() -> back()` **uses session to keep the previous URL** then it redirect to the previous URL.
 - We will learn about session in the next chapter.

```
class ProductController extends SearchableController
{
    // Now we add addShop() method
    function addShop(
        string $productCode,
        ServerRequestInterface $request,
    ): RedirectResponse {
        // method body in on the next page
    }
}
```

Example — products.add-shop (cont.)

3. In ProductController (cont.)

```
// Method body
$product = $this->find($productCode);
$shopController = \resolve(ShopController::class);
$data = $request->getParsedBody();

// To make sure that no duplicate shop.
$shopQuery = $shopController->getQuery();
$this->filterOutShopByProduct($shopQuery, $product);
$shop = $shopQuery
    ->where('code', $data['shop'])
    ->firstOrFail();

// Add $shop to $product
$product->shops()->attach($shop);

return redirect()->back();
```

Example — update routes/web.php

4. Change route products.view-shop to be grouped routes.

```
// ... In ProductController routes group
// Change Route::get('/shops', 'viewShops')->name('view-shops');
// TO:
Route::prefix('/shops')->group(static function(): void {
    Route::get('', 'viewShops')->name('view-shops');
    Route::post('', 'addShop')->name('add-shop');
    Route::get('/add', 'showAddShopsForm')->name('add-shops-form');
});
```

Example — products.add-shops-form

5. Create template `products/add-shops-form.blade.php`.
- It is same as `products.view-shops`.
 - In [section header](#).

```
<!-- For search form, change action/clear to route products.add-shop-form -->

<!-- add form.app-form-add-shop to <nav> -->
<nav>
    <form action="{ route('products.add-shop', [
        'product' => $product->code,
    ]) }}" id="app-form-add-shop" method="post">
        @csrf
    </form>

    <!-- Change back link to products.view-shops -->
</nav>
```

Example — products.add-shops-form (cont.)

6. In table, **add submission buttons** for adding specific shop.
- In **section content**.
 - We use `$shop->code` to refer to added shop.

```
<table>
  <!-- table header -->
  <tbody>
    @foreach($shops as $shop)
      <tr>
        <!-- Other columns -->
        <td>
          <button type="submit"
            form="app-form-add-shop"
            name="shop" value="{{ $shop->code }}">
            Add
          </button>
        </td>
      </tr>
    @endforeach
  </tbody>
</table>
```

Example — update `products.view-shops`

7. Now we add the link to route `products.add-shops-form` in template `products.view-shop.blade.php`.

```
<!-- Link to route products.add-shops-form -->
<a href="{{ route('products.add-shops-form', [
    'product' => $product->code,
]) }}">Add Shops</a>
```

Example — products.remove-shop

8. In ProductController

- We use detach() method to remove shop from product.

```
class ProductController extends SearchableController
{
    // Now we add removeShop() method
    function removeShop(
        string $productCode,
        ServerRequestInterface $request,
    ): RedirectResponse {
        $product = $this->find($productCode);
        $data = $request->getParsedBody();

        // To get existing shop
        $shop = $product->shops()->where('code', $data['shop'])->firstOrFail();

        // Remove $shop from $product
        $product->shops()->detach($shop);

        return redirect()->back();
    }
}
```

Example — update routes/web.php

9. Add all new routes **after** route `products.add-shop`.

```
// ... In prefix /products/{product}/shops  
// ... After route products.add-shops-form  
Route::post('/remove', 'removeShop')->name('remove-shop');
```

Example — update `products.view-shops`

10. Next, we add **form link** for removing shop in template `products.view-shops`.

```
<!-- In <nav> of section header, add form.app-form-remove-shop -->
```

```
<nav>
```

```
  <form action="{{ route('products.remove-shop', [
    'product' => $product->code,
  ]) }}" id="app-form-remove-shop" method="post">
    @csrf
  </form>
```

```
  <!-- other links -->
```

```
</nav>
```

Example — update products.view-shops (cont.)

11. We add **submission buttons** for removing specific shop.
- We use `$shop->code` to refer to removed shop.

```
<!-- In <table> of section content -->
<table>
  <!-- table header -->
  <tbody>
    @foreach($shops as $shop)
      <tr>
        <!-- Other columns -->
        <td>
          <button type="submit"
            form="app-form-remove-shop"
            name="shop" value="{{ $shop->code }}">
            Remove
          </button>
        </td>
      </tr>
    @endforeach
  </tbody>
</table>
```

References

- <https://laravel.com/framework/docs/13.x/eloquent-relationships>